| Projeto | Curso | Turma |
|---|---|---|
| Projeto Interdisciplinar – Aplicativo Desktop para Gestão de Assinaturas de Jogos Educacionais | Análise e Desenvolvimento de Sistemas | 1º Semestre, 2026 |

| Tema | Título |
|---|---|
| Desenvolvimento de um Aplicativo Desktop com acesso a Banco de Dados | Sistema Desktop para Gestão de Assinaturas de Jogos Educacionais – Messier Data & Creative |

Aplicativo Desktop – Assinaturas de Jogos Educacionais (Messier)

Projeto Interdisciplinar – 1ADS – 1º SEMESTRE 2026

## 1. Apresentação do Projeto

A Messier Data & Creative é uma empresa de desenvolvimento de jogos e experiências interativas que combina tecnologia e criatividade para promover engajamento em eventos, demonstrações de produto e campanhas de marca. Como novo passo, a Messier pretende lançar uma plataforma de jogos digitais para escolas de ensino fundamental e médio.

Neste Projeto Interdisciplinar, as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo.

Fluxo do processo (simulação do módulo Escola):

1) Login: a escola se autentica; o aplicativo verifica se a escola possui pacote ativo e se o IP de origem (informado na simulação) está autorizado.

2) Catálogo: o aplicativo apresenta os games disponíveis no pacote adquirido pela escola.

3) Acesso: a escola simula o acesso a um game; o aplicativo registra e contabiliza o acesso no consumo do mês, respeitando o limite do pacote.

## 2. Objetivos

Objetivo Geral:

Desenvolver uma aplicação Desktop com banco de dados para gerenciar assinaturas de jogos educacionais por escolas, incluindo cadastro de games e pacotes, controle de pacote ativo por escola, validação por IP autorizado, contabilização de acessos mensais e relatórios.

Objetivos Específicos:

Aplicar conceitos de algoritmo e lógica de programação na implementação do fluxo de login, validações e contadores de acessos;

Modelar e implementar um banco de dados relacional para armazenar escolas, pacotes, games, IPs autorizados e logs de acesso;

Empregar princípios de modelagem e arquitetura de software para organizar a solução em camadas (UI, regras de negócio e persistência);

Estabelecer a comunicação entre aplicação e banco de dados (CRUD e consultas de relatórios);

Compreender fundamentos de redes ao tratar conceitos de IP, autorização por origem e possíveis extensões cliente-servidor;

Relacionar conceitos de matemática discreta (lógica, relações, conjuntos e grafos) às regras de validação, limites e estados do processo.

## 3. Requisitos Funcionais

Cadastro de Games (CRUD) com nome, descrição, tema/disciplina, faixa etária e status ativo/inativo;

Cadastro de Pacotes (CRUD) com: nome do pacote, lista de games incluídos, limite de acessos/mês e preço mensal;

Cadastro de Escolas (CRUD) com dados básicos, pacote adquirido (apenas um pacote ativo por escola) e lista de IPs autorizados;

Login do módulo Escola com validação de pacote ativo e verificação do IP de origem (informado na simulação);

Exibição dos games disponíveis conforme o pacote ativo da escola;

Registro de acesso a game (log com data/hora, escola, game, IP, resultado permitido/bloqueado) e incremento do contador mensal;

Bloqueio de acesso quando o limite mensal do pacote for atingido, com mensagem explicativa;

Relatório de acessos por escola e por período (ex.: mês/ano), incluindo consumo acumulado;

Relatório de escolas e pacotes (pacote vigente, limite mensal, consumo do mês e percentual de uso).

## 4. Requisitos Não Funcionais

A aplicação deve funcionar offline (modo local) para fins de protótipo;

Interface intuitiva e compatível com desktop (Windows), com telas de lista + formulário;

Tempo de resposta adequado para consultas e relatórios com base de dados pequena/média;

Código modular e organizado em camadas (UI, regras de negócio, dados/persistência);

Persistência em banco relacional com integridade (chaves, relacionamentos e validações);

Documentação clara e padronizada do projeto e do código (README e manual rápido de execução).

## 5. Tecnologias Sugeridas

Linguagem/Plataforma: C# (.NET);

Interface Desktop: WinForms (recomendado para início) ou WPF;

Banco de Dados: SQLite;

Acesso a dados: ADO.NET (recomendado) ou Entity Framework Core (se aplicável);

Ferramentas de Modelagem: Draw.io, StarUML ou Lucidchart;

Controle de Versão: Git (GitHub ou GitLab);

Prototipação de interface: Figma/Canva ou recursos nativos.

## 6. Etapas do Projeto

1. Semana 1-2: Apresentação do desafio, formação de equipes e entendimento das regras de negócio;

2. Semana 3-4: Levantamento de requisitos, protótipo de telas e casos de uso;

3. Semana 5-6: Modelagem do banco de dados (conceitual e físico) e criação do projeto base;

4. Semana 7-8: Implementação do MVP do módulo Messier (cadastros de games, pacotes e escolas);

5. Semana 9-10: Implementação do módulo Escola (login, validação por IP, catálogo e registro de acesso/contador);

6. Semana 11-12: Relatórios, testes, ajustes de usabilidade e validações de limite mensal;

7. Semana 13: Entrega e apresentação final (demo) e documentação.

## 7. Possíveis Extensões da Solução

Dashboard simples com gráficos de consumo mensal por escola/pacote;

Notificação de alerta ao atingir 80% do limite mensal;

Exportação de relatórios (CSV/PDF) e importação de escolas/IPs (CSV);

Perfis de acesso (Administrador/Escola) com auditoria de tentativas de login;

Simulação cliente-servidor (API local) para aproximar de um cenário real de plataforma;

Obs: Cada equipe pode propor e implementar suas próprias extensões.

# Entregáveis

| Unidades Curriculares | **Entrega 1**<br>**(3,0 Pontos)**<br>**06/04 23:59:56** | **Entrega 2**<br>**(4,0 Pontos)**<br>**11/05 23:59:56** | **Apresentação**<br>**(3,0 Pontos) Semana FECAP de Tecnologia**<br>**21.22.25.26 e 27 de Maio** |
|---|---|---|---|
| Fundamentos de Banco de Dados | Formato de Entrega: Modelagem Conceitual do Banco de Dados<br><br>Objetivo: Os alunos devem realizar a modelagem conceitual do banco de dados para o projeto. A modelagem conceitual deve representar as entidades e seus relacionamentos, usando diagramas como o Diagrama Entidade-Relacionamento (DER), que será documentado no arquivo README da pasta BD. Um link para esse arquivo deve ser inserido no README principal do projeto. | Formato de Entrega: Modelagem Física de Tabelas<br><br>Objetivo: Os alunos devem realizar a modelagem física do banco de dados, criando pelo menos duas tabelas com base no modelo conceitual previamente definido. As tabelas devem ser documentadas no arquivo README da pasta BD. O link para esse README deve ser inserido no README principal do projeto | **Formato de entrega: Demonstração (demo) + banner e pitch de até 4 minutos.**<br><br>**Critérios: Criatividade, impacto social, qualidade técnica, clareza, métricas e rastreabilidade.** |
| Redes de Computadores | Formato de Entrega: Simulação de Verificação de IP (C#)<br>Objetivo: Implementar e demonstrar a validação de acesso por IP (origem informada na simulação), incluindo casos 'autorizado' e 'bloqueado', registrando log de tentativas. | Formato de Entrega: Packet Tracer + Documento (Word/PDF)<br>Objetivo: Projetar uma topologia de rede para uma escola (laboratórios, secretaria e Wi-Fi), com segmentação básica (sub-redes ou separação lógica), endereçamento IPv4 e justificativa de boas práticas. | |

| | | | |
|---|---|---|---|
| Matemática Discreta | **Entrega 1**<br><br>**Formato de Entrega:** Documento (PDF) — “Regras em Lógica”<br><br>**Objetivo:** Formalizar, usando Lógica Proposicional, as regras do sistema (permitir/bloquear acesso). O grupo deve:<br><br>• Definir de 4 a 6 proposições (ex.: A = “pacote ativo”, I = “IP autorizado”, L = “limite mensal não atingido”, P = “acesso permitido”);<br>• Escrever a regra principal em forma lógica (ex.:<br>• $P \leftrightarrow (A \wedge I \wedge L)$ ou equivalente);<br>• Montar a tabela-verdade da regra e identificar em quais casos o acesso é permitido/bloqueado;<br>• Criar 3 cenários (situação → valores de A, I, L → conclusão P) explicando em 1 linha cada. | **Entrega 2**<br><br>**Formato de Entrega**: Lista de Exercícios de Probabilidade (PDF)<br><br>**Objetivo:** Aplicar probabilidade básica em situações do projeto (acessos, autorizações e limites), com interpretação simples. O grupo deve:<br><br>• Definir eventos e espaço amostral para 3 situações do sistema (ex.: “acesso permitido”, “IP autorizado”, “limite atingido”);<br>• Resolver 4 questões de probabilidade (pode usar: complemento, união/interseção e condicional);<br>• Incluir 1 questão de probabilidade condicional (ex.: $P$(Permitido\|IP autorizado)<br>• Finalizar com 3–5 linhas explicando como os resultados ajudam a pensar regras/risco de bloqueio/uso. | |
| Algoritmos e Lógica de Programação | Instruções de Entrega: Aplicação das estruturas sequencial, de decisão e de repetição no PI. Entregar um documento PDF com o nome do projeto, o professor orientador e os nomes dos integrantes do grupo. No documento deve conter a parte da aplicação das estruturas sequencial, de decisão e de repetição (proibido colocar todo o código do PI) e os seus resultados aplicados no projeto (determinando a entrada e a saída), bem como, observações reflexivas do grupo. Proibido o uso de short cuts ou atalhos matemáticos. O documento PDF deve ficar dentro de uma pasta denominada Algoritmos e Lógica de Programação no github do projeto.<br><br>Objetivo: Capacidade de os estudantes aplicarem algoritmos em projetos reais, sabendo observar e definir | Instruções de Entrega: Aplicação das estruturas de modularização, vetores e matrizes no PI. Entregar um documento PDF com o nome do projeto, o professor orientador e os nomes dos integrantes do grupo. No documento deve conter a parte da aplicação das estruturas de modularização, vetores e matrizes (proibido colocar todo o código do PI) e os seus resultados aplicados no projeto (determinando a entrada e a saída), bem como, observações reflexivas do grupo. Proibido o uso de short cuts ou atalhos matemáticos. O documento PDF deve ficar dentro de uma pasta denominada Algoritmos e Lógica de Programação no github do projeto.<br><br>Objetivo: Capacidade de os estudantes aplicarem algoritmos em projetos reais, sabendo observar e definir mudanças se os | |

| | | | |
|---|---|---|---|
| | mudanças se os resultados estiverem divergentes ao esperado, melhorando a entrega do PI. | resultados estiverem divergentes ao esperado, melhorando a entrega do PI. | |
| Modelagem de Software e Arquitetura de Sistemas | Instruções de Entrega: Documento de abertura do projeto, Contextualização e objetivo da solução, Identificação de Requisitos funcionais e não funcionais.<br><br>Descrição: Identificar 6 requisitos funcionais e 6 não funcionais do sistema | Instruções para Entrega: Casos de Uso e Diagrama de Classe do Sistema<br><br>Descrição: Os alunos devem criar um Diagrama de Classe que represente as principais entidades do sistema e seus relacionamentos. Esse diagrama deve refletir a estrutura do sistema em termos de classes, atributos, métodos e os relacionamentos entre as classes. Construir 3 casos de uso.<br><br>Instruções para Entrega: Projeto de Arquitetura da Implementação do Sistema<br><br>Descrição: Os alunos devem elaborar um Projeto de Arquitetura que descreva a estrutura e a organização do sistema, abrangendo desde as camadas de apresentação até a persistência de dados. O projeto de arquitetura deve detalhar como as diferentes partes do sistema interagem e como a implementação será organizada de forma modular, escalável e eficiente. | |
| Detalhes do projeto: | Tecnologias que utilizam: C# | | |

# Cronograma de Atividades/Tarefas

| ATIVIDADE/TAREFA | NOTA | PENALIDADE | DATA LIMITE |
|---|---|---|---|
| **Cadastro/formação do time (grupos definitivos; regras de composição)** | — | Cadastros indevidos/fora das regras: -1,0 ponto do grupo. | 10/Mar |
| **Aceitar o convite do repositório no GitHub (ver e-mail/SPAM)** | — | Convite expira; se expirar e precisar reenvio, pode gerar desconto (ver abaixo). | Até 7 dias após o envio do convite |
| **Solicitar reenvio do convite (se o convite expirar)** | — | -1,0 ponto na nota final por cada reenvio; se pedir reenvio com semanas de atraso: -0,25 ponto por semana adicional. | Após expirar (quanto antes, melhor) |
| **Entrega 1 (E1) via GitHub/GitHub Project** | 3,0 pts | Atraso: perda de 25% da nota total por dia. | 06/04 (até 23:59:56) |
| **Enviar o Banner no GitHub para aprovação (pasta 'DOCUMENTOS')** | — | Podemos reduzir até 25% da nota da apresentação para banners não aprovados. | Até 04/05 |
| **Avaliação 360° (entre pares)** | — (multiplicador 0,0 a 1,0) | Pode reduzir a nota final (fator multiplicador definido pela avaliação entre pares). | 09/05 (2 dias antes de 11/05) |
| **Entrega 2 + Documento de extensão COM EMPRESA - Final (E2) via GitHub/GitHub Project** | 4,0 pts | Atraso: perda de 25% da nota total por dia. | 11/05 (até 23:59:56) |
| **Enviar o Banner para a gráfica (TJ Impressos) (PDF + comprovante PIX)** | — | Envio após o prazo: aumento de valor e risco de não produção a tempo. | Prazo máximo: 14/05 |
| **Sistema Final (SF) - apenas ADS4 (sistema funcional com front/back/comunicação)** | 1,0 pt | Envio após o prazo pode até zerar a nota | 11/05 (até 23:59:56) |
| **Apresentação (Pitch) + Banner** | 3,0 pts | Sem banner: pode reduzir em até 25% a nota da apresentação. | Semana FECAP de Tecnologia 21,22,25,26 e 27 de Maio |